Glossário OAuth 2.0

Grazielly Gonçalves Lacerda

Parte 1

Back end

`google-auth-library`

É a biblioteca oficial desenvolvida pelo próprio Google para Node.js. Ela fica no back-end para que o nosso servidor consiga validar a autenticação do Google de forma direta e segura, sem precisarmos construir a lógica de verificação manual.

import { OAuth2Client } from 'google-auth-library'

`OAuth2Client`

É a classe fornecida pela biblioteca para criar a instância do cliente de autenticação. Passamos o `GOOGLE_CLIENT_ID` para que o servidor saiba exatamente qual aplicação registrada no Google Cloud está executando essa validação.

const googleClient = new OAuth2Client(process.env.GOOGLE_CLIENT_ID)

`verifyIdToken()`

É o método que valida se o token enviado pelo front-end é autêntico, se pertence à nossa aplicação e se ainda está no prazo de validade. Se o token for falso ou estiver alterado, a função lança um erro que cai no bloco `catch`, cancelando o login e retornando o status `401`.

const ticket = await googleClient.verifyIdToken({

idToken: credential,

audience: process.env.GOOGLE_CLIENT_ID})

`credential` (ID Token)

É a credencial recebida no corpo da requisição (req.body). Esse valor é gerado no front-end pelo próprio componente do Google assim que o usuário faz o login, sendo então enviado ao nosso servidor para ser autenticado.

const { credential } = req.body

`audience`

É a propriedade onde informamos novamente o nosso `GOOGLE_CLIENT_ID` durante a checagem. Ela garante que o token recebido foi gerado especificamente para a nossa aplicação, impedindo que um token válido emitido para outro site consiga acessar o nosso sistema.

const ticket = await googleClient.verifyIdToken({

idToken: credential,

audience: process.env.GOOGLE_CLIENT_ID})

`getPayload()`

É o método que extrai as informações do usuário contidas dentro do token validado.

- **O que devolve:** Um objeto com os dados cadastrais do Google.
- **4 campos principais:** `sub`, `email`, `nome` e `foto`.

const dadosGoogle = ticket.getPayload()

`sub`

É o identificador único e permanente do usuário dentro do Google. Usamos o `sub` para identificar a conta no banco de dados porque, ao contrário do e-mail (que o usuário pode alterar nas configurações do Google), o `sub` nunca muda.

const [resultado] = await db.query(

*"SELECT * FROM usuarios WHERE google_id = ? OR email = ?",*

[dadosGoogle.sub, dadosGoogle.email])

`email_verified`

É uma propriedade informando se o e-mail daquela conta Google já foi confirmado pelo proprietário. Recusamos o login caso seja `false` para impedir o cadastro de contas não verificadas que poderiam comprometer a segurança.

if (!dadosGoogle.email_verified) {

return res.status(401).json({ mensagem: "Email do Google não foi verificado" })}

Nosso JWT (`gerarToken`)

O token do Google serve apenas para confirmar quem é o usuário no momento do login. Uma vez confirmada a identidade, o nosso servidor gera um token JWT próprio contendo os dados internos do nosso banco (como o `id` e o `role`). Com isso, as outras rotas do sistema (como a rota de perfil) autenticam o usuário usando a nossa própria assinatura, sem precisar consultar o Google a cada requisição.

const { token, payload } = await gerarToken(usuario)

return res.status(200).json({

mensagem: "Login com o google realizado com sucesso",

token,

usuario: payload })

Front end

`@react-oauth/google`

É uma biblioteca de React que facilita a inclusão do botão de login e do fluxo de autenticação do Google no front-end. **Não é mantida oficialmente pelo Google** (é um projeto de código aberto da comunidade), mas utiliza por baixo dos panos a biblioteca e os scripts oficiais do Google (*Google Identity Services*).

`GoogleOAuthProvider`

No React, um **Provider** é um componente que utiliza a API de Contexto (*Context API*) para disponibilizar dados e configurações para todos os componentes dentro dele, sem precisar passar variáveis manualmente de filho para filho.

O `GoogleOAuthProvider` envolve o `<App/>` para carregar o script do Google e disponibilizar a chave `clientId` para qualquer botão ou gancho (*hook*) de login do Google dentro da aplicação.

`clientId / VITE_GOOGLE_CLIENT_ID`

- **Client ID:** É uma chave **pública**. Ela serve apenas para o Google saber qual projeto está pedindo o login. Como qualquer pessoa consegue ver o código do front-end no navegador, não há problema em expor o Client ID.
- **Client Secret:** É uma chave **privada** de alta segurança. Se ela ficasse no front-end, um usuário mal-intencionado poderia abri-la no código-fonte e usar a sua conta de desenvolvedor do Google. Por isso, o Secret **nunca** deve ir para o front-end.

`GoogleLogin`

É o componente pronto de botão fornecido pela biblioteca. Quando o usuário clica nele, o componente abre a janela oficial do Google para que o usuário escolha uma conta e confirme o acesso. Assim que o Google autoriza, o botão recebe a credencial e dispara a função onSuccess.

`onSuccess / onError`

- **`onSuccess`:** É chamado quando o usuário faz login com sucesso no pop-up do Google. Ele recebe um objeto com a propriedade `credential` (o ID Token do Google). Esse é o valor enviado ao back-end.
- **`onError`:** É chamado se houver qualquer falha (se o usuário fechar a janela, se a conexão cair ou se as configurações do Client ID estiverem erradas).

`googleLogout()`

Limpar o `localStorage` apaga o token do *nosso* sistema que estava salvo no navegador. Porém, a biblioteca do Google pode manter em memória interna o estado de sessão da conta do usuário.

A função `googleLogout()` limpa o estado ativo da biblioteca do Google no navegador, garantindo que o ciclo de login/logout seja completamente encerrado antes de uma nova tentativa.